

1. Core

1.1. Core Library

`heroLib_core` is the foundation every other Hero library builds on. It is a general-purpose Rust crate with no async runtime requirement and no daemon dependencies; a set of small, sharp tools you can pull in by feature flag.

1.1.1. When to use which module

- Use `base` in any binary or library that needs to know where Hero roots, repos., or sockets live. Most Hero binaries start with `service_base()`.
- Use `logger` when you want fast, local, append-only log files without installing a database or broker.
- Use `secrets` in library code that needs a credential but must not assume a supervising daemon; it reads an injected value if one was set, and falls back to the environment otherwise.

1.1.2. specialty

- Use `heroscript` when you want human-editable configuration that is still safe to load and process automatically. It is a good fit for deployment descriptors, job definitions, or any file a human writes and a program acts on e.g. in markdown.
- Use `heroscript::text` when you need to edit those files back in place without rewriting the whole file.

1.1.3. Quick example

A minimal Hero binary typically starts like this:

```
heroLib_core::service_base();

fn main() {
    println!("repo manifest: {:?}", REPO_REPO_PATH);
    println!("build number: {:?}", BUILD_NUM);
    println!("base root: {:?}", heroLib_core::base::path_root());
}
```

From there you can load HeroScript and write logs using the modules in the following chapters.